

Software Architecture and Design Specification

Project: Electronics Store Inventory Management System

Group 16

Version: 1.0

Team : Shreyas Gowda K A, Shreya C Rao, R Laksheeth

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Group 16	Initial SASD for ESIMS

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Electronics Store Inventory Management System (ESIMS). It explains how the main parts of the system are organized and how the important features will work.

1.2 Scope

The system is a web-based inventory system for a small or medium electronics store. It supports secure login, product addition, product deletion, inventory search, quantity updates, automatic stock-status classification, availability reports, and a dashboard showing current inventory information.

1.3 Audience

Developers, project team members, testers, project guide, and other people who need to understand the design of the system.

1.4 Definitions

Term	Meaning
ESIMS	Electronics Store Inventory Management System
Admin	User who has permission to manage products and user access
Inventory	Current products and their available quantities
Reorder Level	Quantity level used to identify low stock
API	Interface used for communication between the web interface and backend

2. Document Overview

2.1 How to use this document

This document gives a simple overview of the ESIMS architecture and design. It includes the main components, database design, important system flows, APIs, security, user interface, and design decisions.

2.2 Related Documents

- Software Requirements Specification (SRS)
- Project Proposal
- Software Test Plan (STP), if prepared

- Requirement Traceability Matrix (RTM), if prepared

3. Architecture

3.1 Goals & Constraints

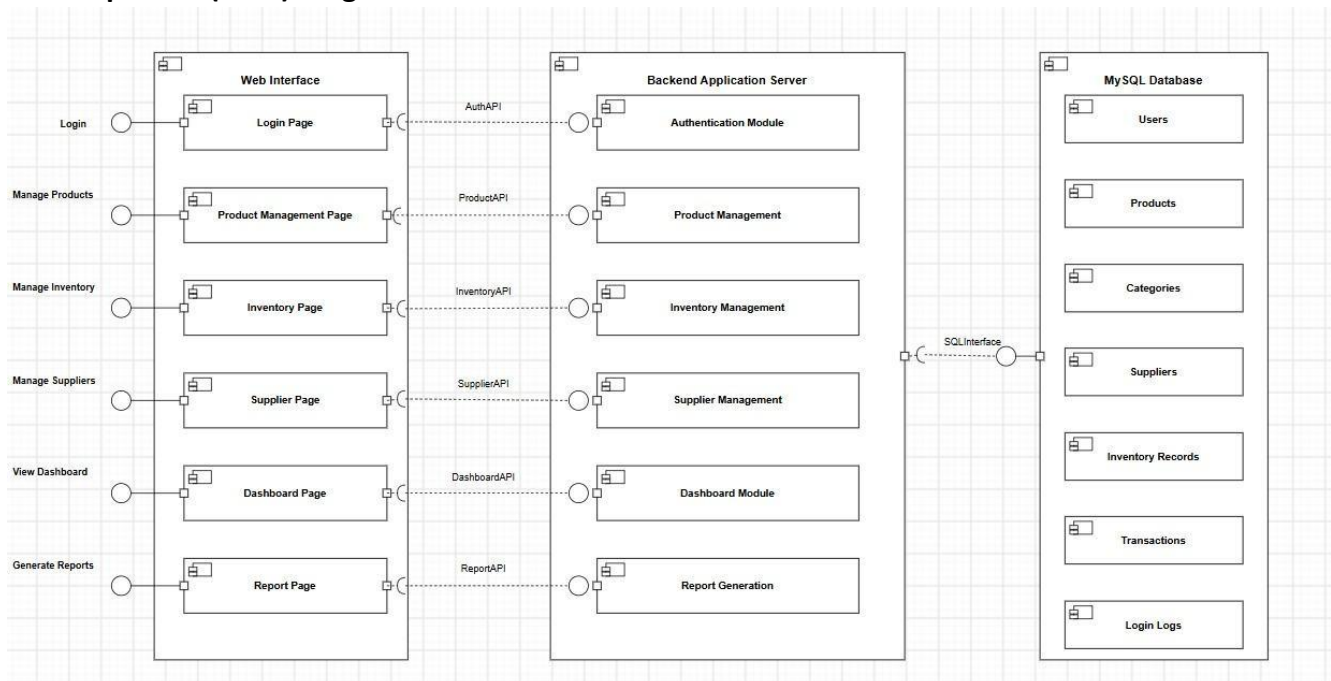
Goals: provide a simple, secure, and easy-to-use inventory system; keep current stock information accurate; make searching and filtering quick; and keep the design easy for the team to implement and maintain.

Constraints: this is an academic project with a limited team and development time. Version 1.0 maintains the current available quantity and does not maintain detailed stock-in/stock-out transaction history.

3.2 Stakeholders & Concerns

Stakeholder	Main Concern
Administrator	Login security, product management, access control
Store Manager	Accurate inventory, reports, dashboard
Store Employee	Easy product search and quantity updates
Management	Clear current stock information
Developers/Team	Simple implementation and maintainability

3.3 Component (UML) Diagram



3.4 Component Descriptions

- Web Interface: provides login, dashboard, inventory, product forms, search, and report screens.
- Login & Access Control: checks user credentials and controls access according to the user's role.
- Inventory Management: adds, updates, searches, and deletes product information.
- Stock Status Logic: classifies products as In Stock, Low Stock, or Out of Stock using quantity and reorder level.

- Backend / API: receives requests from the interface, applies rules, and communicates with the database.
- Database: stores users, products, categories, and current inventory information.

3.5 Chosen Architecture Pattern and Rationale

A simple layered architecture is chosen. The interface, application logic, and database are kept as separate parts. This makes the project easier to understand, test, modify, and demonstrate. A more complex architecture is unnecessary for the size and scope of this academic project.

3.6 Technology Stack & Data Stores

Part	Proposed Technology
Frontend	HTML, CSS, JavaScript
Backend	Python Flask / REST API
Database	MySQL
Development	VS Code, Git

3.7 Risks & Mitigations

Risk	Mitigation
Duplicate product ID	Validate the product ID before adding a product.
Incorrect quantity	Validate input and allow only valid quantity values.
Unauthorized access	Use login and role-based access.
Database errors	Validate requests and show clear error messages.
Team integration issues	Use Git and integrate each member's work regularly.

3.8 Traceability to Requirements

Requirement	Design Component
Login and role-based access	Login & Access Control
Add/Delete Product	Inventory Management
Search/View Inventory	Inventory Management
Update current quantity	Inventory Management
Automatic stock classification	Stock Status Logic
Availability report	Report Module
Dashboard	Dashboard Module

3.9 Security Architecture

Users must log in before accessing protected inventory features. Access is controlled according to user roles. Input validation is used for product details and quantities. Passwords should be stored securely rather than as plain text. Database access should be restricted to the backend, and sensitive information should not be exposed in error messages or logs.

4. Design

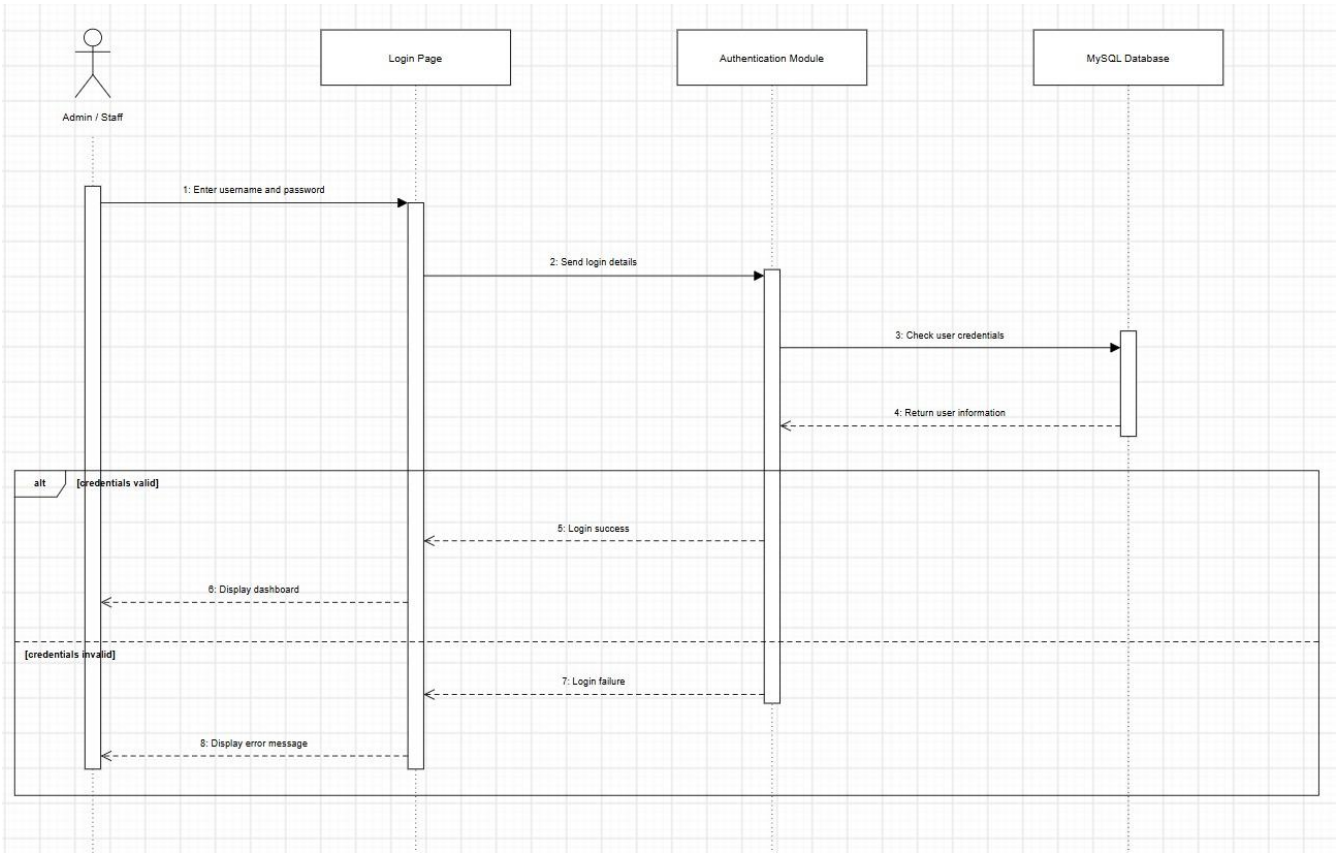
4.1 Design Overview

ESIMS is designed as a small layered web application. A user interacts with the web interface. The backend receives the request, checks the required rules, and reads or updates the MySQL database. The result is then displayed to the user.

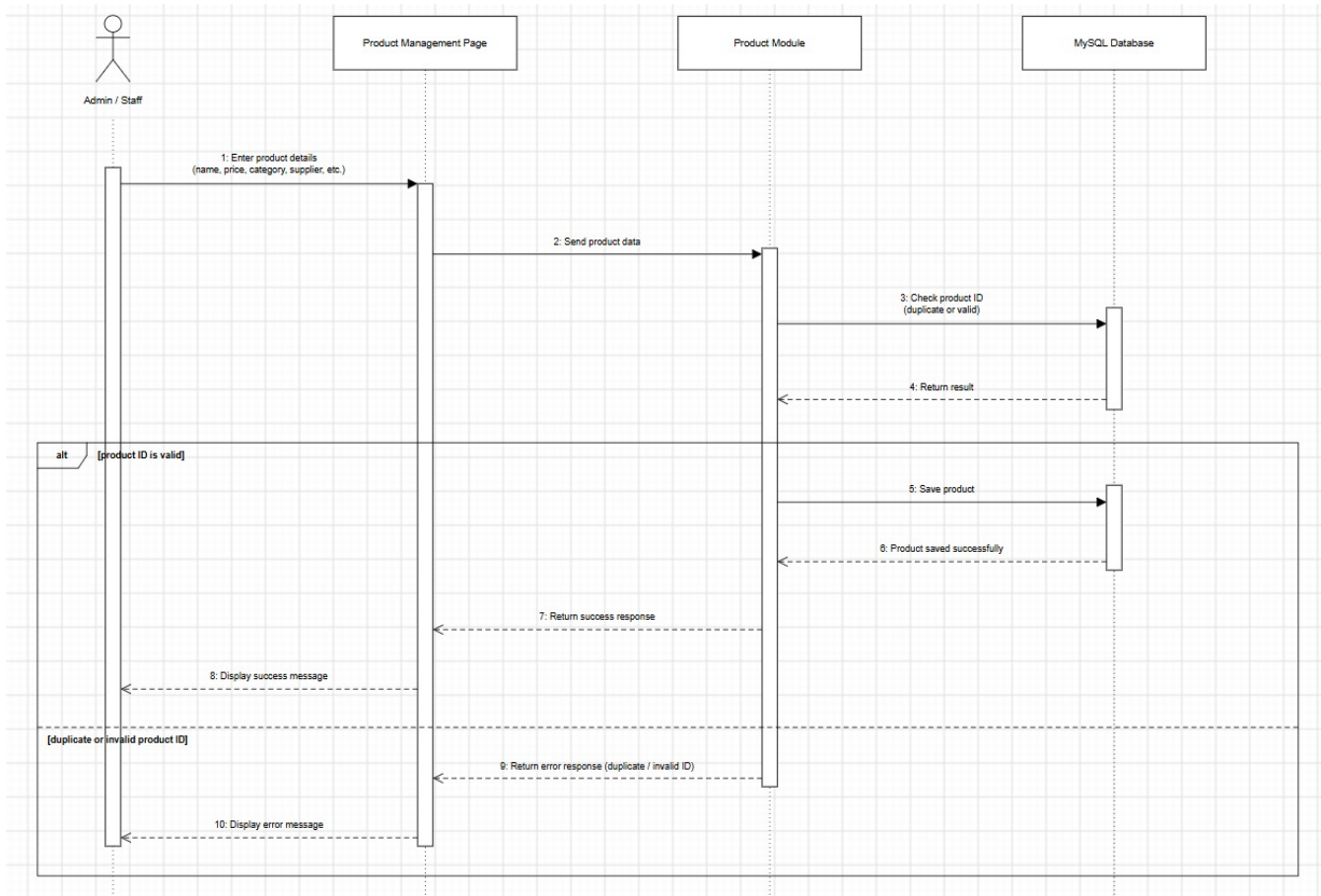
4.2 UML Sequence Diagrams

At least two important flows are shown below.

Sequence 1: Login



Sequence 2: Add Product



4.3 API Design

The following are example interfaces between the web interface and backend.

Endpoint	Method	Purpose
/login	POST	Validate user login details
/products	GET	View/search products
/products	POST	Add a new product
/products/{id}	PUT	Update product details or current quantity
/products/{id}	DELETE	Delete an existing product
/reports/availability	GET	Get products grouped/filterable by stock status

Example product data: { productId, name, category, price, quantity, reorderLevel, location }

Common errors include invalid input, duplicate product ID, product not found, invalid login details, and unauthorized access.

4.4 Error Handling, Logging & Monitoring

The system should show simple and clear error messages to the user. Examples include 'Invalid login details', 'Product ID already exists', and 'Product not found'. The system should record useful application errors for debugging without storing passwords or other sensitive information.

4.5 UX Design

The interface should be simple and easy to use. The main screens are Login, Dashboard, Inventory, Add/Edit Product, and Availability Report. Search and filters should be clearly visible. Forms should show validation messages when required information is missing or incorrect.

4.6 Open Issues & Next Steps

- Complete implementation and integration of all team modules.
- Perform unit and integration testing.
- Connect the dashboard and reports to current database values.
- Test role-based access and input validation.
- Possible future enhancement: maintain detailed stock-in/stock-out history.
- Possible future enhancement: add stronger reporting and export options.

5. Appendices

5.1 Glossary

Term	Meaning
ESIMS	Electronics Store Inventory Management System
API	Application Programming Interface
UML	Unified Modeling Language
SRS	Software Requirements Specification
CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control

5.2 References

- Project SRS and proposal documents
- Project Deliverable-3 SASD reference/template
- Course material for Software Engineering

5.3 Tools

- VS Code
- Git/GitHub
- MySQL
- Python/Flask
- Draw.io or PlantUML for diagrams